8 Roteiro Prático 07 — Introdução ao JavaScript e Validação de Formulários

Objetivos da Aula

Ao final desta aula o estudante será capaz de:

- Compreender o papel do JavaScript na web;
- Capturar eventos de formulário;
- Manipular elementos HTML com JavaScript;
- Criar validações personalizadas;
- Exibir mensagens dinâmicas ao usuário.

Introdução

Até agora, utilizamos apenas HTML e CSS para estruturar e estilizar o site.

Nesta aula, iremos introduzir o JavaScript, que permite adicionar comportamento e interação à página.

Vamos utilizar o formulário criado anteriormente como base para implementar validações personalizadas.

Preparação do Ambiente

```
git pull
```

Parte 1 — Criando o Arquivo JavaScript

Crie um novo arquivo:

```
assets/js/script.js
```

Agora, conecte o arquivo ao HTML:

```
<script src="assets/js/script.js"></script>
```

Coloque essa linha antes do fechamento da tag `</body>`.

Parte 2 — Capturando o Evento de Envio

```
const form = document.querySelector("form");

form.addEventListener("submit", function(event) {
    event.preventDefault();
    console.log("Formulário enviado!");
});
```

Dicas Úteis — VS Code

O método `preventDefault()` impede o recarregamento da página.

Parte 3 — Lendo os Campos

```
const nome = document.getElementById("nome").value;
const email = document.getElementById("email").value;

console.log(nome);
console.log(email);
```

Parte 4 — Criando Validação

```
if (nome.length < 3) {
    alert("O nome deve ter pelo menos 3 caracteres.");
    return;
}

if (!email.includes("@")) {
    alert("Email inválido.");
    return;
}
```

Parte 5 — Feedback ao Usuário

```
alert("Formulário enviado com sucesso!");
```

Parte 6 — Exibindo Mensagem na Página

Adicione no HTML:

```
<p id="mensagem-sucesso"></p>
```

E no JavaScript:

```
document.getElementById("mensagem-sucesso").innerText =
    "Formulário enviado com sucesso!";
```

Resultado Final Esperado

- O formulário não recarrega a página;
- Os dados são capturados corretamente;
- Validações são aplicadas;
- Mensagens são exibidas ao usuário.

Versionamento

```
git add .
git commit -m "Adiciona validação com JavaScript"
git push
```

Tabela de Referência — JavaScript

Comando	Exemplo	Função
querySelector	document.querySelector("form	Seleciona elemento
addEventListener	form.addEventListener(...)	Escuta eventos
preventDefault	event.preventDefault()	Impede envio padrão
getElementById	document.getElementById("no	Seleciona por ID
value	input.value	Obtém valor do campo
alert	alert("mensagem")	Exibe alerta
innerText	element.innerText	Altera texto na página
includes	email.includes("@")	Verifica conteúdo

Desafio Extra

- Validar telefone com JavaScript;
- Exibir mensagem personalizada sem usar alert;
- Limpar o formulário após envio;
- Criar validação mais avançada.

9 Roteiro Prático 08 — Introdução ao JavaScript e Consumo de API

Objetivos da Aula

Ao final desta aula o estudante será capaz de:

- Entender o funcionamento básico do JavaScript;
- Manipular elementos da página;
- Trabalhar com eventos;
- Capturar dados de inputs;
- Consumir uma API utilizando fetch.

Introdução

Nesta aula, teremos o primeiro contato com JavaScript.

Antes de trabalhar com APIs, vamos explorar como o JavaScript pode modificar a página em tempo real.

Parte 1 — Criando uma Página de Teste

Crie um novo arquivo:

```
teste-js.html
```

9.3.1 HTML base

```
<h1 id="titulo">Minha Página</h1>

<button id="btnAlerta">Mostrar alerta</button>
<button id="btnCor">Mudar cor</button>

<script src="assets/js/script.js"></script>
```

Parte 2 — Exibindo Alertas

```
document.getElementById("btnAlerta")
    .addEventListener("click", function() {
        alert("Olá! JavaScript funcionando!");
});
```

Parte 3 — Alterando Estilos

```
document.getElementById("btnCor")
    .addEventListener("click", function() {

    document.body.style.backgroundColor = "lightblue";

});
```

Atividade 1

- Alterar a cor para outra cor;
- Testar diferentes valores;
- Modificar também a cor do texto.

Parte 4 — Modificando Conteúdo

```
document.getElementById("titulo").innerText =
    "Texto alterado com JavaScript!";
```

Exploração — Propriedades e Eventos JavaScript

A tabela abaixo apresenta algumas propriedades e eventos que podem ser utilizados para modificar elementos da página.

Recurso	Exemplo	Descrição / Possibilidades
innerText	element.innerText = "Novo texto"	Altera o texto do elemento. Pode ser usado para mensagens dinâmicas.
style.color	element.style.color = "red"	Altera a cor do texto. Testar: red, blue, green, black
style.backgroundColor	document.body.style.backgrou = "lightblue"	Altera a cor de fundo. Testar: yellow, pink, gray
style.fontSize	element.style.fontSize = "20px"	Altera o tamanho da fonte. Testar: 12px, 18px, 30px
style.border	element.style.border = "2px solid black"	Adiciona borda ao elemento. Testar diferentes cores e espessuras
value	input.value	Captura o valor digitado pelo usuário
alert	alert("Mensagem")	Exibe uma mensagem em popup
console.log	console.log(valor)	Exibe informações no console do navegador
click	addEventListener("click", ...)	Executa ação ao clicar em um elemento
mouseover	addEventListener("mouseover" ...)	Executa ação ao passar o mouse sobre o elemento
mouseout	addEventListener("mouseout", ...)	Executa ação ao retirar o mouse do elemento
input	addEventListener("input", ...)	Executa ação enquanto o usuário digita
change	addEventListener("change", ...)	Executa ação ao alterar um campo (ex: select)

Explorando JavaScript

- Alterar a cor de fundo da página;
- Modificar o texto do título;
- Criar um efeito ao passar o mouse;
- Capturar o valor digitado em um campo;
- Exibir mensagens personalizadas.

Parte 5 — Aplicando no Projeto

Agora vamos utilizar JavaScript no formulário criado anteriormente.

```
const botao = document.getElementById("buscarCep");

botao.addEventListener("click", function() {
    console.log("Botão clicado!");
});
```

Parte 6 — Buscando Dados

Substitua o código anterior:

```
const botao = document.getElementById("buscarCep");

botao.addEventListener("click", function() {

    const cep = document.getElementById("cep").value;

    fetch(‘https://viacep.com.br/ws/${cep}/json/‘)
        .then(response => response.json())
        .then(dados => {

            if (dados.erro) {
                document.getElementById("resultadoCep").innerText =
                    "CEP não encontrado.";
                return;
            }

            document.getElementById("resultadoCep").innerText =
                dados.logradouro + " - " + dados.localidade;

        });

});
```

Resultado Final

- JavaScript alterando a página;
- Eventos funcionando;
- Dados sendo buscados da internet;
- Integração com formulário.

Dicas Úteis — VS Code

Como funciona o código com fetch?
Vamos entender passo a passo o que está acontecendo:

- **fetch(url)** Realiza uma requisição para um servidor na internet. No nosso caso, estamos consultando a API do ViaCEP.
- **response** É a resposta do servidor. Ainda não é o dado final, apenas a resposta da requisição.
- **response.json()** Converte a resposta para o formato JSON (JavaScript Object). Agora podemos acessar os dados como `dados.logradouro`, `dados.localidade`, etc.
- **.then(...)** Define o que deve acontecer após a resposta chegar. Como a requisição leva um tempo, usamos `then` para continuar o código depois.
- **=> (arrow function)** É uma forma simplificada de escrever funções em JavaScript.

  Exemplo:

  ```
  (dados) => { ... }
  ```

- **Fluxo geral do código**

  1. O usuário clica no botão;
  2. O JavaScript captura o valor do CEP;
  3. O `fetch` envia a requisição;
  4. O servidor responde com os dados;
  5. O `.then()` recebe esses dados;
  6. O JavaScript atualiza a página com o resultado.

Resumo: O JavaScript pede dados → o servidor responde → o código trata a resposta → a página é atualizada.

Tabela de Referência — JavaScript Utilizado

Comando	Exemplo	O que faz
getElementById	document.getElementById("cep")	Seleciona um elemento pelo ID
addEventListener	botao.addEventListener("click", fn)	Executa função ao clicar
function	function() {...}	Define uma função
fetch	fetch(url)	Faz requisição para internet
then	.then(...)	Executa após resposta
response.json	response.json()	Converte resposta para JSON
template string	‘$cep‘	Insere variável dentro da URL
innerText	elemento.innerText	Altera texto na tela
if	if(condição) {...}	Estrutura de decisão
return	return	Interrompe execução
console.log	console.log(dados)	Mostra dados no console